

Tecnología Digital VI

Problemas sobre imágenes - I2I

Licenciatura en Tecnología Digital - UTDT

Más allá de la clasificación

A partir del salto que comenzó en el 2012* se pudieron resolver problemas sobre imágenes que antes parecían intratables o muy difíciles. En general, se atacaban con técnicas muy específicas para cada problema, y con datasets acotados.

La revolución de las CNNs hizo que el problema de clasificación de imágenes pasara a ser muy tratable en casi todos los datasets. Sin embargo, seguimos hablando de clasificar una imagen en una categoría como el único gran problema a resolver.

¿Si una foto tiene 4 perros y 4 gatos, qué estamos esperando que diga la red?
¿Qué otros problemas sobre imágenes nos podrían interesar resolver que no sean puramente una única clasificación?

Más allá de la clasificación

Ahora que tenemos estos formalismos potentes (y encima preentrenados!) podemos pensar en varios otros problemas sobre imágenes como:

- Detectar *dónde* está el objeto.
- Detectar muchos objetos.
- Partir las imágenes en los diferentes objetos presentes y en el fondo.
- Describir una imagen en texto.
- Y muchos más

Más allá de la clasificación

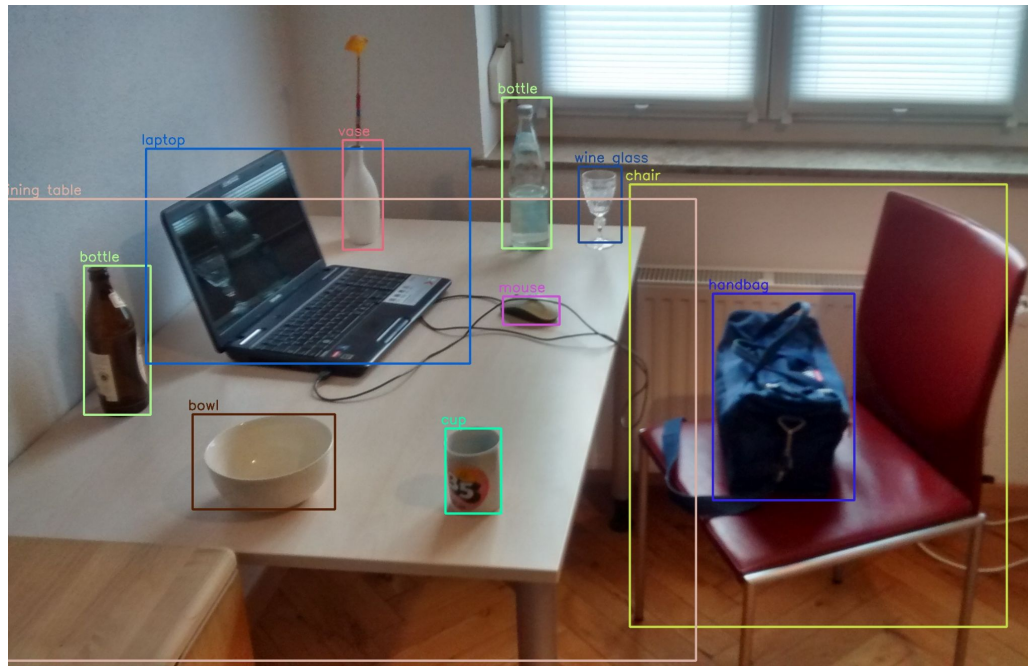
La idea es buscar reutilizar todo lo aprendido hasta el momento para resolver otros problemas, incluso al punto de reutilizar literalmente las mismas redes pero aplicándolas de diferentes maneras.

Hubo y hay muchas ideas innovadoras para reutilizar y readaptar estos formalismos. Algunas familias de ideas posibles:

- Tomar el formalismo como ya existe, pero aplicarlo de formas diferentes.
- Reutilizar las ideas principales del formalismo pero cambiar ciertas componentes como la capa de salida, la loss function, etc.

Object Detection

El problema de Object Detection consta de no solo clasificar los objetos de una imagen, sino de poder localizarlos en la misma mediante la inserción de una *bounding box*.



Object Detection

Tengo un martillo, hagamos que todo sea un clavo.

Si tengo una red ya entrenada que predice muy muy bien 1000 tipos de objetos ¿cómo la uso para resolver mi problema de Object Detection?

Una posible idea es hacer una *sliding window* sobre la imagen y en cada parche hacer inferencia con una ResNet.

Es una idea razonable, pero con varios problemas:

- Los objetos tienen tamaños muy distintos, ¿qué tamaño de *sliding window* uso?
- Si uso tamaños muy chicos (y si uso más de un tamaño), el costo computacional puede ser prohibitivo aún haciendo solo inferencia.

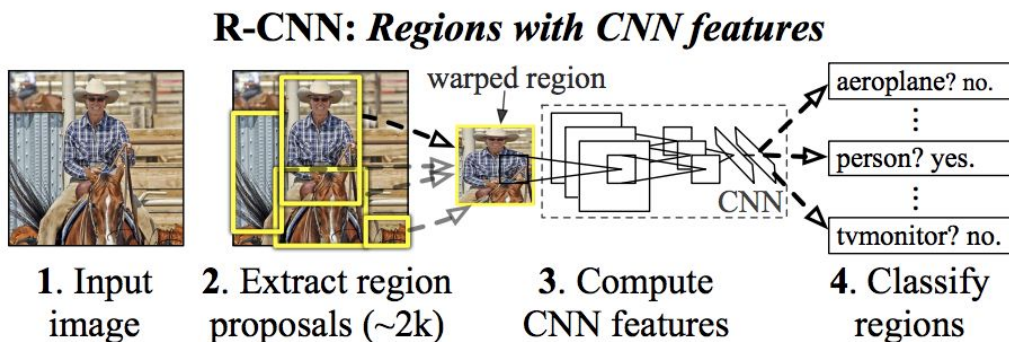
R-CNN

La familia de modelos R-CNN (Region-Based CNN) para Object Detection aplicó esta misma idea pero de forma más inteligente para sobrellevar los problemas planteados.

En vez de simplemente pasar por todos los posibles parches de la imagen, primero se buscaba generar unas 2000 regiones candidatas.

Luego, sobre esas regiones, se aplicaban clasificaciones individuales.

Si bien era una idea muy superadora a la *sliding window* seguía siendo muy lenta.



Fast R-CNN y Faster R-CNN

En los años subsiguientes a R-CNN salieron nuevos modelos como Fast R-CNN y Faster R-CNN que buscan mitigar el mayor problema que tienen estas familias *two-stage* que es el tiempo de ejecución.

En Fast R-CNN se mejoran los tiempos de ejecución evitando tratar cada región como algo completamente independiente.

En Faster R-CNN se reemplaza el algoritmo de propuesta de regiones por otra red (Region Proposal Network).

Las familias R-CNN siguen siendo modelos vigentes e importantes. No tienen mucho uso en donde el *real-time* es un feature importante, pero sí en aplicaciones donde prima la detección robusta.

En paralelo al desarrollo de la familia R-CNN, nació el modelo YOLO (You Only Look Once) con la particularidad de buscar resolver el problema con una sola pasada por la imagen.

La diferencia sustancial de YOLO es que ya no vamos a tomar nuestro martillo así como viene, sino que vamos a modificarlo.

Más allá de Regresión y Clasificación

Hasta ahora vimos problemas de Regresión y de Clasificación. En redes neuronales eso se tradujo de la siguiente manera:

- Regresión: que la capa de salida sea un solo output.
- Clasificación: que la capa de salida sean tantos outputs como clases y busquemos (mediante la loss function) interpretar esas salidas como probabilidades de pertenencia.

¿Quién dijo que estas son las dos únicas cosas que podemos hacer con una red? Justamente la versatilidad de las redes es que puedo *escupir* cuantos valores yo quiera. Luego, será responsabilidad de la loss function y de un conjunto de entrenamiento correctamente anotado el darle semántica a cada una de esas salidas para que tengan sentido práctico y resuelvan el problema que tenemos.

YOLO v “-1”

Pensemos en el caso más simple de que estamos seguros de que hay exactamente un objeto en mis imágenes. Adicionalmente, tenemos la suerte de que el objeto presente sabemos que pertenece a una de la 1000 clases de ImageNet.

Una primera aproximación a nuestro problema de Object Detection podría ser la siguiente:

- Me siento a anotar todas mis imágenes. Por cada imagen anoto a qué clase pertenece el objeto y las coordenadas x e y de la *bounding box*.
- Defino una red muy similar a ResNet34 (por decir alguna), pero la última capa además de tener 1000 salidas para las clases posibles, va a tener 4 salidas más que representan los x e y de las bounding boxes.
- Defino una loss function que penalice, como ya sabemos, la pertenencia a las clases incorrectas, pero que también penalice cuando la bounding box predicha se aleja de la real.
- Entreno sobre todas mis imágenes anotadas.

La primera versión de YOLO en el 2015 aplicó algo similar a la idea presentada, pero soportando el hecho de que una imagen puede tener muchos objetos.

You Only Look Once: Unified, Real-Time Object Detection

Joseph Redmon*, Santosh Divvala*[†], Ross Girshick[‡], Ali Farhadi*[†]

University of Washington*, Allen Institute for AI[†], Facebook AI Research[‡]

<http://pjreddie.com/yolo/>

Abstract

We present YOLO, a new approach to object detection. Prior work on object detection repurposes classifiers to perform detection. Instead, we frame object detection as a regression problem to spatially separated bounding boxes and associated class probabilities. A single neural network predicts bounding boxes and class probabilities directly from full images in one evaluation. Since the whole detection pipeline is a single network, it can be optimized end-to-end directly on detection performance.

Our unified architecture is extremely fast. Our base YOLO model processes images in real-time at 45 frames per second. A smaller version of the network, Fast YOLO, processes an astounding 155 frames per second while still achieving double the mAP of other real-time detectors. Compared to state-of-the-art detection systems, YOLO makes more localization errors but is less likely to predict false positives on background. Finally, YOLO learns very general representations of objects. It outperforms other detection methods, including DPM and R-CNN, when generalizing from natural images to other domains like artwork.

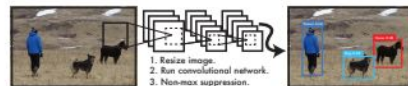


Figure 1: The YOLO Detection System. Processing images with YOLO is simple and straightforward. Our system (1) resizes the input image to 448×448 , (2) runs a single convolutional network on the image, and (3) thresholds the resulting detections by the model's confidence.

methods to first generate potential bounding boxes in an image and then run a classifier on these proposed boxes. After classification, post-processing is used to refine the bounding boxes, eliminate duplicate detections, and rescore the boxes based on other objects in the scene [13]. These complex pipelines are slow and hard to optimize because each individual component must be trained separately.

We reframe object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. Using our system, you only

2. Unified Detection

We unify the separate components of object detection into a single neural network. Our network uses features from the entire image to predict each bounding box. It also predicts all bounding boxes across all classes for an image simultaneously. This means our network reasons globally about the full image and all the objects in the image. The YOLO design enables end-to-end training and real-time speeds while maintaining high average precision.

Our system divides the input image into an $S \times S$ grid. If the center of an object falls into a grid cell, that grid cell is responsible for detecting that object.

Each grid cell predicts B bounding boxes and confidence scores for those boxes. These confidence scores reflect how confident the model is that the box contains an object and also how accurate it thinks the box is that it predicts. Formally we define confidence as $\Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}}$. If no object exists in that cell, the confidence scores should be zero. Otherwise we want the confidence score to equal the intersection over union (IOU) between the predicted box and the ground truth.

Each bounding box consists of 5 predictions: x, y, w, h , and confidence. The (x, y) coordinates represent the center of the box relative to the bounds of the grid cell. The width and height are predicted relative to the whole image. Finally the confidence prediction represents the IOU between the predicted box and any ground truth box.

Each grid cell also predicts C conditional class probabilities, $\Pr(\text{Class}_i | \text{Object})$. These probabilities are conditioned on the grid cell containing an object. We only predict

one set of class probabilities per grid cell, regardless of the number of boxes B .

At test time we multiply the conditional class probabilities and the individual box confidence predictions,

$$\Pr(\text{Class}_i | \text{Object}) * \Pr(\text{Object}) * \text{IOU}_{\text{pred}}^{\text{truth}} = \Pr(\text{Class}_i) * \text{IOU}_{\text{pred}}^{\text{truth}} \quad (1)$$

which gives us class-specific confidence scores for each box. These scores encode both the probability of that class appearing in the box and how well the predicted box fits the object.

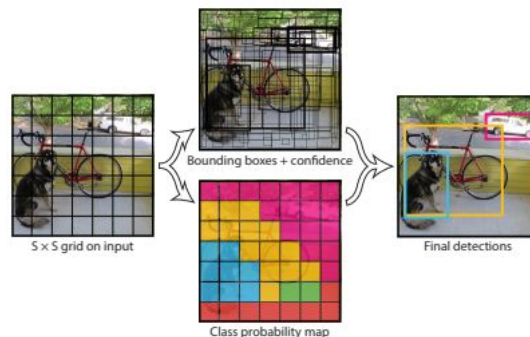


Figure 2: The Model. Our system models detection as a regression problem. It divides the image into an $S \times S$ grid and for each grid cell predicts B bounding boxes, confidence for those boxes, and C class probabilities. These predictions are encoded as an $S \times S \times (B * 5 + C)$ tensor.

For evaluating YOLO on PASCAL VOC, we use $S = 7$, $B = 2$. PASCAL VOC has 20 labelled classes so $C = 20$. Our final prediction is a $7 \times 7 \times 30$ tensor.

YOLO v1 – Arquitectura

La arquitectura en sí no tiene mucho misterio, es una reimplementación de GoogLeNet pero, esencialmente, modificando la última capa.

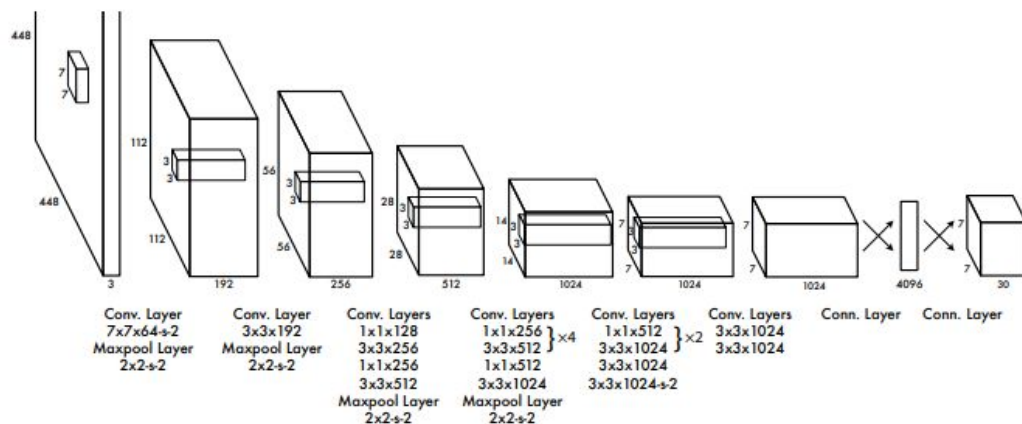


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

YOLO v1 – Loss Function

La gran adaptación de YOLO viene por el lado de la loss function.

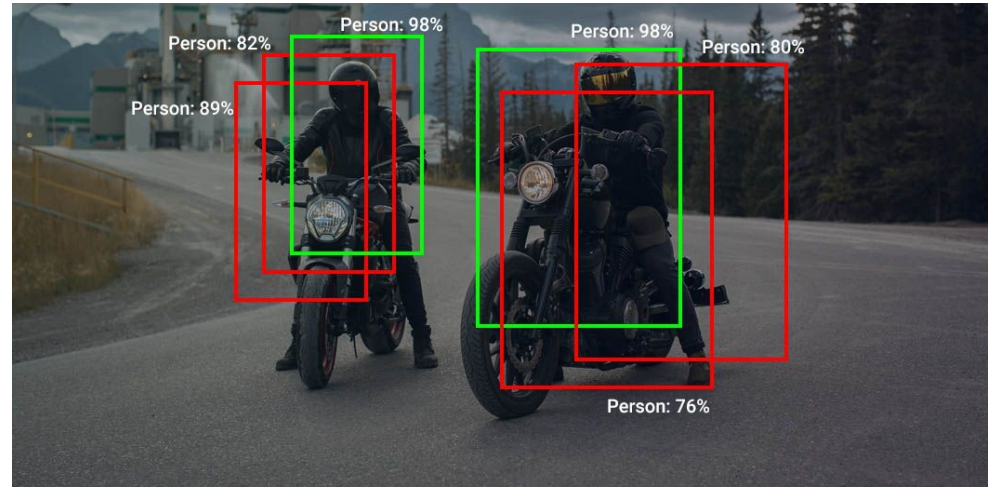
A primera vista parece compleja, pero no es más que un gran error cuadrático de cada cosa que nos importa (las funciones indicadoras prenden y apagan lo que nos importa en cada caso):

- Primer término: Penaliza si la red erra el centro de la bounding box.
- Segundo término: Penaliza el tamaño incorrecto de la bounding box.
- Tercer término: Penaliza la diferencia en la confianza de la caja.
- Cuarto término: Penaliza la confianza al no haber objetos.
- Quinto término: Penaliza una mala clasificación de clase.

$$\begin{aligned} & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\ & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\ & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\ & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\ & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2 \end{aligned}$$

YOLO v1 – Non-Max Suppression

Por como se separa en regiones la imagen, podría suceder que varias celdas de la división quieran hacerse cargo del mismo objeto. YOLO v1 tiene como último paso una iteración de *non-max suppression* que es un algoritmo simple para descartar bounding boxes que tengan mucha IoU con otra bounding box de mayor confianza.



YOLO v1 – Resultados

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45

Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Compar-

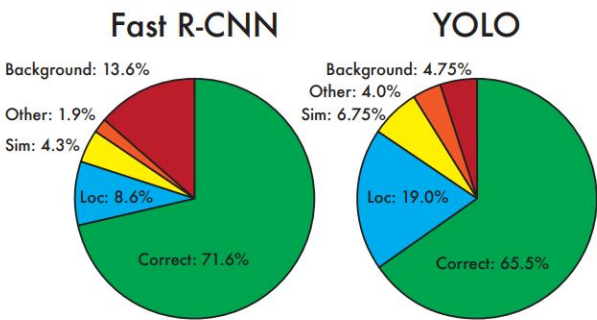
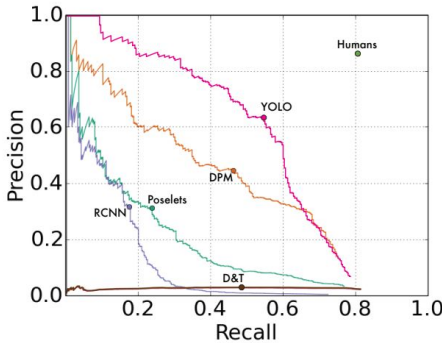


Figure 4: Error Analysis: Fast R-CNN vs. YOLO These charts show the percentage of localization and background errors in the top N detections for various categories (N = # objects in that category).



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso AP	Picasso Best F_1	People-Art AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 5: Generalization results on Picasso and People-Art datasets.

La explosión de YOLO

Con el correr de los años se desarrollaron diferentes versiones de YOLO (YOLO v2, v3, ..., v11, v12, YOLO26) que fueron mejorando sustancialmente los resultados de la primera versión, transformándose en el estandar de la industria para Object Detection (sobre todo para problemas *real-time*).

Adicionalmente, la idea de modificar la loss-function convenientemente, sirvió para derivar modelos que no solo hacen Object Detection, sino también detección de pose o tracking.

Model	Filenames	Task	Inference	Validation	Training
YOLO26	yolo26n.pt yolo26s.pt yolo26m.pt yolo26l.pt yolo26x.pt	Detection	✓	✓	✓
YOLO26-seg	yolo26n-seg.pt yolo26s-seg.pt yolo26m-seg.pt yolo26l-seg.pt yolo26x-seg.pt	Instance Segmentation	✓	✓	✓
YOLO26-pose	yolo26n-pose.pt yolo26s-pose.pt yolo26m-pose.pt yolo26l-pose.pt yolo26x-pose.pt	Pose/Keypoints	✓	✓	✓
YOLO26-obb	yolo26n-obb.pt yolo26s-obb.pt yolo26m-obb.pt yolo26l-obb.pt yolo26x-obb.pt	Oriented Detection	✓	✓	✓
YOLO26-cls	yolo26n-cls.pt yolo26s-cls.pt yolo26m-cls.pt yolo26l-cls.pt yolo26x-cls.pt	Classification	✓	✓	✓

YOLO – Ejemplo

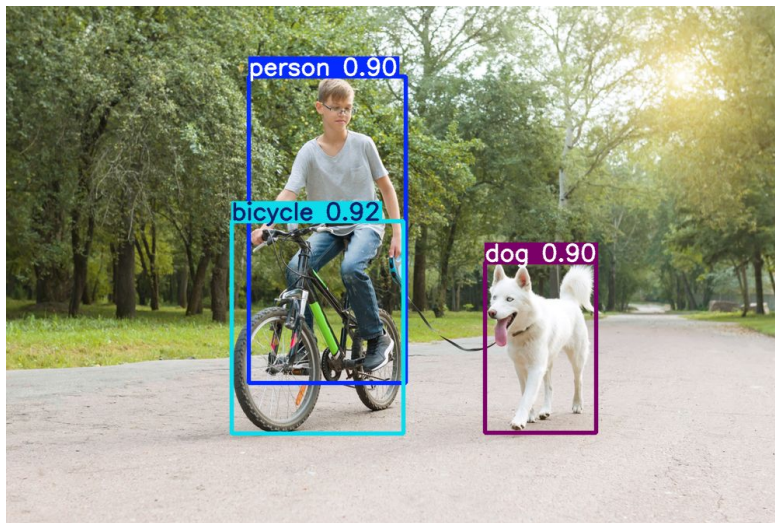
```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
resultados = model('dogbike.jpg')

resultados[0].show()
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to 'yolov8n.pt': 100% ————— 6.2MB 13.4MB/s 0.5s.4s<0.1s3s

image 1/1 /home/fpousa/Desktop/Fede/UTDT/LTD/TD6/Codigos/NN/dogbike.jpg: 448x640 1 person, 1 bicycle, 1 dog, 29.3ms
Speed: 3.5ms preprocess, 29.3ms inference, 19.3ms postprocess per image at shape (1, 3, 448, 640)



Ya modificamos nuestras redes para que arrojen más outputs que solamente un valor de regresión o varios valores de probabilidades para clasificar.

¿Y si hacemos que arrojen muchos más outputs?

¿Y si hacemos que arrojen tantos outputs como inputs recibieron?

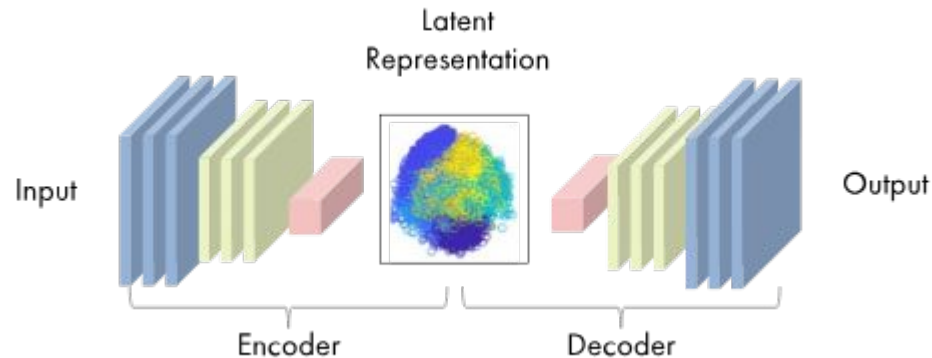
Depende qué sean esos outputs, incluso podríamos estar *generando* una nueva imagen en la salida.

Nacen así los problemas I2I (Image to Image).

Autoencoders

Arranquemos con un primer caso muy simple. Una red convolucional que arroja exactamente la misma imagen que entró.

Esta red, denominada autoencoder, está conformada primero por un Encoder (capas convolucionales que van achicando la cantidad de parámetros) y luego por un Decoder (capas convolucionales que vuelven a agrandar la cantidad de parámetros hasta obtener la imagen original).



Autoencoders

Esta simple arquitectura ya trae un montón de preguntas:

- ¿Cuál es la loss function?
- ¿Por qué primero achica y luego agranda? ¿No pueden simplemente mantener todas el tamaño?
- ¿Qué es eso de *agrandar*? Las convoluciones que vimos hasta ahora achicaban.
- ¿Para qué puede servir esto?

Autoencoder – Loss Function

Para el autoencoder más simple que solo funciona como la identidad, la loss function puede ser algo tan directo como el error cuadrático medio pixel a pixel (aunque se podrían usar otras métricas de qué tanto se parece la imagen a la original).

Con esto se destaca que no hay ninguna *ground truth* contra la que comparar más que el input mismo. Es decir, no hay que etiquetar nada! Es similar a lo que nos sucedía en el aprendizaje no-supervisado. En este caso se le suele llamar *self-supervised learning*.

Autoencoder – Down y Upsampling

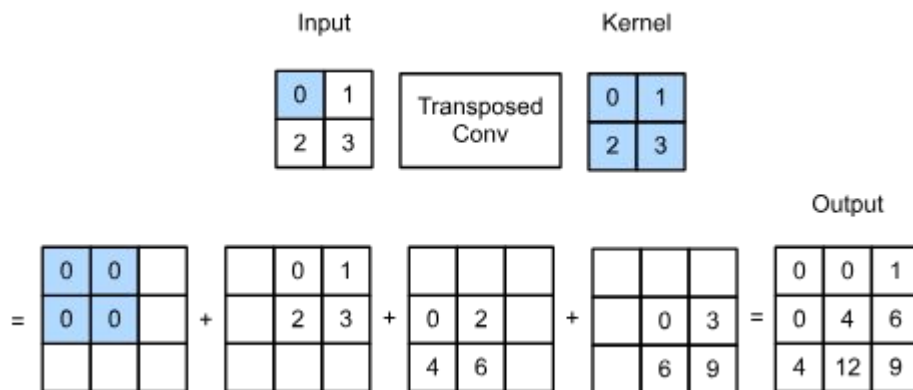
¿Por qué esta arquitectura primero va achicando el tamaño de la imagen y luego agrandándolo?

- En primer lugar, serían muchísimos parámetros si cada capa conserva el tamaño.
- En segundo lugar, el campo receptivo de cada filtro sería exactamente el mismo siempre en todas las capas (y encima muy chiquitos), no hay manera de aprender nada interesante.
- Por último, en el caso de esta loss function en particular, si mantuviesen el tamaño sería trivial hacer que la red sea la identidad.

Autoencoder – Upsampling

Las convoluciones vistas hasta ahora achicaban el espacio.

Se definió otra operación denominada Convolución Transpuesta que lo que hace es a partir de un feature map de cierto tamaño (ej 2x2) crea otro de tamaño más grande (ej 3x3) con un filtro aprendible (o sea, con parámetros) de 2x2.



Autoencoder – Aplicaciones

Así como lo definimos, ¿para qué puede servir?

- Reducción de la dimensionalidad: Si nos quedamos solo con el Encoder, tenemos un “PCA no lineal”.
- Image retrieval: Para medir si dos imágenes son parecidas, el espacio latente suele funcionar muy bien.
- Denoising: Le pasamos una imagen con ruido agregado, pero la loss function es contra la imagen original. La red aprende a hacer Denoising.
- Inpainting: Igual que denoising pero el agregado ahora es poner un parche negro en la imagen.
- Detección de anomalías: Si entrenamos con muchas muestras *normales*, cuando venga una anómala va a tener un error de reconstrucción mucho más grande.

Denoising

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Entrenando en: {device}")
```

```
transform = transforms.ToTensor()
train_dataset = torchvision.datasets.MNIST(root='./data', train=True, download=True, transform=transform)
train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
```

```
class AutoencoderDenoising(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1),
            nn.ReLU()
        )
        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(16, 1, kernel_size=3, stride=2, padding=1, output_padding=1),
            nn.Sigmoid() # Salida entre 0 y 1 (escala de grises)
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))
```

```
modelo = AutoencoderDenoising().to(device)
criterio = nn.MSELoss()
optimizador = optim.Adam(modelo.parameters(), lr=0.005)
```

```
epochs = 3
```

```
epochs = 3
```

```
print("Iniciando entrenamiento...")
for epoch in range(epochs):
    loss_total = 0
    for imagenes_perfectas, _ in train_loader:
        imagenes_perfectas = imagenes_perfectas.to(device)

        ruido = torch.randn_like(imagenes_perfectas) * 0.3
        imagenes_rotas = torch.clamp(imagenes_perfectas + ruido, 0., 1.)

        imagenes_predichas = modelo(imagenes_rotas)

        loss = criterio(imagenes_predichas, imagenes_perfectas)

        optimizador.zero_grad()
        loss.backward()
        optimizador.step()

    loss_total += loss.item()

    print(f"Epoch [{epoch+1}/{epochs}], Loss Promedio: {loss_total/len(train_loader):.4f}")

print("¡Entrenamiento terminado!")
```

Denoising

Entrenando en: cuda

```
100% | 9.91M/9.91M [00:03<00:00, 3.06MB/s]
100% | 28.9k/28.9k [00:00<00:00, 166kB/s]
100% | 1.65M/1.65M [00:02<00:00, 690kB/s]
100% | 4.54k/4.54k [00:00<00:00, 3.21MB/s]
```

Iniciando entrenamiento...

Epoch [1/3], Loss Promedio: 0.0132

Epoch [2/3], Loss Promedio: 0.0063

Epoch [3/3], Loss Promedio: 0.0062

¡Entrenamiento terminado!

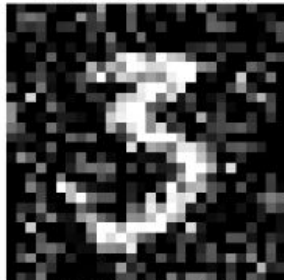
Entrada (Con Ruido)



Salida (Limpia)



Entrada (Con Ruido)



Salida (Limpia)



Entrada (Con Ruido)



Salida (Limpia)



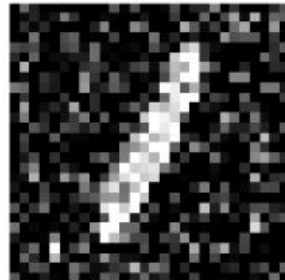
Entrada (Con Ruido)



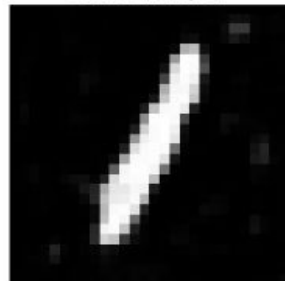
Salida (Limpia)



Entrada (Con Ruido)

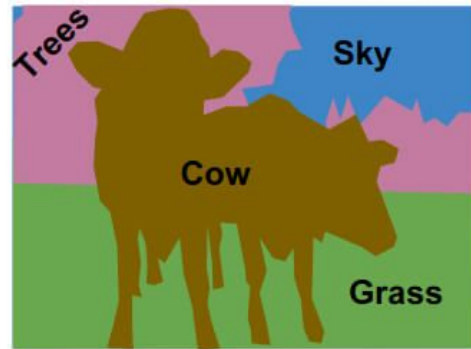
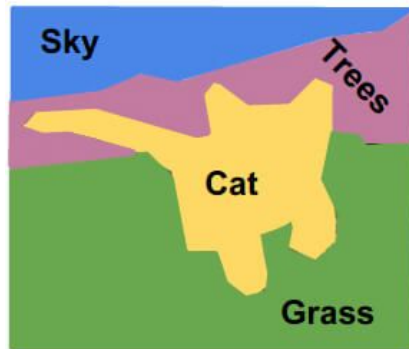


Salida (Limpia)



Segmentación semántica

Lo que sale de este tipo de redes aunque tiene “tamaño de imagen” no necesariamente tiene que ser una imagen en el sentido clásico. El problema de Segmentación Semántica busca traducir cada píxel de la imagen original en una clasificación que indique a qué clase de objeto de la imagen pertenece.



Segmentación: Loss Function

¿Qué podemos utilizar como Loss Function para que la red clasifique bien cada píxel?

La cross-entropy clásica no sirve mucho porque no utiliza la localidad espacial del propio output (manchas conexas).

Existen otras Loss Functions como Dice Loss, Jaccard Loss, Focal Loss que justamente buscan explotar la localidad espacial de lo que predicen.

Dice Loss por ejemplo es la formalización del concepto de Intersection over Union. El coeficiente Dice es una relación entre la intersección y la unión de lo predicho y la *ground truth*.

$$\text{Dice} = \frac{2 \times |X \cap Y|}{|X| + |Y|}$$

Segmentación: Arquitectura

U-Net: Convolutional Networks for Biomedical Image Segmentation

Olaf Ronneberger, Philipp Fischer, and Thomas Brox

Computer Science Department and BIOSS Centre for Biological Signalling Studies,
University of Freiburg, Germany

ronneber@informatik.uni-freiburg.de,

WWW home page: <http://lmb.informatik.uni-freiburg.de/>

Abstract. There is large consent that successful training of deep networks requires many thousand annotated training samples. In this paper, we present a network and training strategy that relies on the strong use of data augmentation to use the available annotated samples more efficiently. The architecture consists of a contracting path to capture context and a symmetric expanding path that enables precise localization. We show that such a network can be trained end-to-end from very few images and outperforms the prior best method (a sliding-window convolutional network) on the ISBI challenge for segmentation of neuronal structures in electron microscopic stacks. Using the same network trained on transmitted light microscopy images (phase contrast and DIC) we won the ISBI cell tracking challenge 2015 in these categories by a large margin. Moreover, the network is fast. Segmentation of a 512x512 image takes less than a second on a recent GPU. The full implementation (based on Caffe) and the trained networks are available at <http://lmb.informatik.uni-freiburg.de/people/ronneber/u-net>.

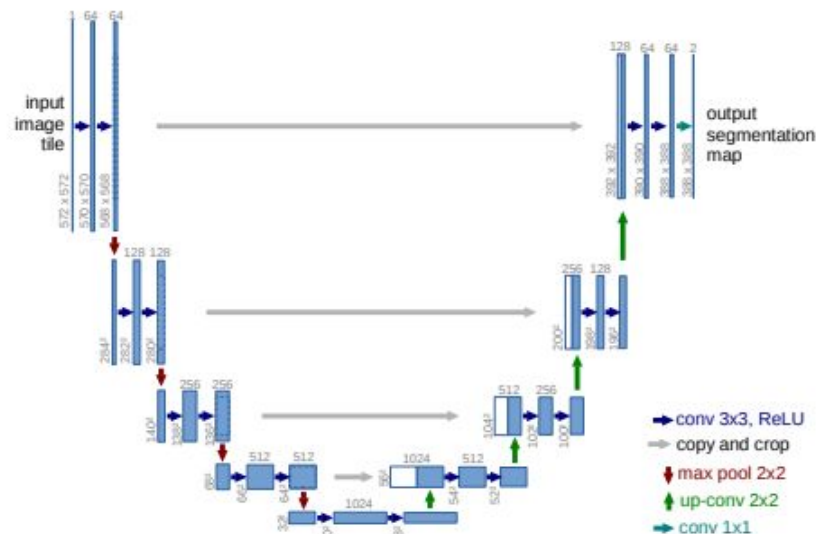


Fig. 1. U-net architecture (example for 32x32 pixels in the lowest resolution). Each blue box corresponds to a multi-channel feature map. The number of channels is denoted on top of the box. The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps. The arrows denote the different operations.

Segmentación: Aplicación

Arrocera Argentina con inspección manual de secones mediante imágenes satelitales.

Se utilizaron arquitecturas y loss functions clásicas de image segmentation para entrenar sobre cientos de imágenes históricas.



FastForward al 2026

Vimos varios problemas diferentes y cómo fueron evolucionando. ¿Qué siguió cambiando hasta el 2026 en estos problemas?

Más allá de las diferentes mejoras que se hicieron a los modelos, la gran diferencia fue la irrupción de los Visual Transformers (ViTs). Una arquitectura basada en los Transformers para texto que comenzó a tener muy buenos resultados en diferentes problemas de imágenes. De todas maneras, esto no significa que las Redes Convolucionales no se utilicen ni muchísimo menos.

- Clasificación: ResNet, ViT, ConvNeXt.
- Object Detection: YOLO26, DETR.
- Segmentación: Mask R-CNN, SAM (Segment Anything Model).

Los otros grandes cambios vinieron por el lado de la explosión de problemas diferentes, el zero-shot learning y la multimodalidad.